Phase 5: Data Modeling and Schema Design Completion Report

1. Phase Overview

Phase 5 of the **Banking Data Governance & Data Quality Pipeline** focused on curated warehouse modeling and schema design.

The goal of this phase was to use standardized Parquet outputs from the Phase 4 PySpark standardized layer and design/build curated warehouse-style fact and dimension models. This phase represents the movement from standardized operational data into analytics-ready curated data models.

The Phase 5 processing flow is:

PostgreSQL banking source system
→ Python ingestion layer
→ raw Parquet zone
→ PySpark standardized processing layer
→ standardized Parquet zone
→ curated warehouse layer

Phase 5 focused only on:

- Curated warehouse modeling
- Fact and dimension design
- Schema design
- Grain definition
- Logical primary key and foreign key design
- Business purpose documentation
- Reading standardized Parquet as input
- Writing curated Parquet outputs
- Lightweight row-count and relationship sanity checks
- README command updates

Phase 5 intentionally did not include SQL data quality validation, PySpark data quality rule execution, OpenMetadata setup, governance YAML artifacts, lineage YAML files, Agile/JIRA simulation files, or final catalog artifacts. Those items belong to later phases.

---

2. Phase 5 Objective

The objective of Phase 5 was:

To design and build curated warehouse-style fact and dimension models from standardized banking data using PySpark, while documenting table grain, logical keys, relationships, source tables, business purpose, and output structure.

Expected Phase 5 input pattern:

data/standardized/<table_name>/load_date=YYYY-MM-DD/<table_name>.parquet

Expected Phase 5 output pattern:

data/curated/<model_name>/load_date=YYYY-MM-DD/<model_name>.parquet

Example:

data/curated/dim_customer/load_date=2026-06-16/dim_customer.parquet

data/curated/fact_transaction/load_date=2026-06-16/fact_transaction.parquet

In local PySpark execution, each .parquet output path is a Spark output directory containing Parquet part files.

---

3. Phase 5 Scope

Phase 5 included the curated warehouse modeling layer.

In Scope

Phase 5 included:

- Curated fact and dimension model design
- Central model configuration
- Business purpose documentation
- Grain definition
- Logical primary key design
- Logical foreign key design
- Target column definition
- Relationship documentation
- PySpark curated model build job
- Standardized Parquet reading
- Curated Parquet writing
- Row-count sanity checks
- Duplicate logical key checks

- Basic null foreign key counts
- Join-loss checks where applicable
- Summary CSV reporting
- README updates
- Git commits

Out of Scope

The following items were intentionally excluded from Phase 5:

- SQL validation scripts
- Formal SQL data quality execution
- PySpark data quality rule framework
- OpenMetadata setup
- Governance YAML artifacts
- Business glossary
- Catalog YAML files
- Final lineage YAML files
- Agile/JIRA simulation files
- Stewardship issue automation

This scope control kept Phase 5 focused on data modeling and curated warehouse construction.

---

4. Curated Models Built

Phase 5 designed and built the following seven curated models:

1. dim_customer
2. dim_account
3. dim_branch
4. dim_product
5. fact_transaction
6. fact_customer_risk_score
7. fact_daily_account_balance

These models represent a simplified banking warehouse layer designed for analytics, governance practice, and future data quality validation.

---

5. Phase 5A: Curated Warehouse Model Design and Configuration

Phase 5A Goal

The goal of Phase 5A was to define the curated warehouse model design before implementation.

Phase 5A did not build curated Parquet outputs. It focused only on model design, configuration, and documentation.

Phase 5A Files Changed

The following files were created or updated:

spark/utils/model_config.py

docs/data_model_design.md

README.md

.gitignore did not need changes because data/curated/ was already ignored.

Phase 5A Work Completed

Phase 5A created a central model configuration file:

spark/utils/model_config.py

This file defines configuration for exactly seven curated models:

- dim_customer
- dim_account
- dim_branch
- dim_product
- fact_transaction
- fact_customer_risk_score
- fact_daily_account_balance

Each model configuration includes:

- Model name
- Model type
- Business purpose

- Grain
- Source tables
- Logical primary key
- Logical foreign keys
- Target columns
- Notes where needed

Phase 5A Documentation

Phase 5A also created:

docs/data_model_design.md

This document explains:

- Phase 5A scope
- Data modeling approach
- Curated model list
- Business purpose for each model
- Grain for each model
- Logical primary keys
- Logical foreign keys
- Source standardized tables
- Target columns
- Relationship overview
- Notes about Parquet constraint limitations
- Reminder that formal SQL DQ belongs to Phase 6
- Reminder that PySpark DQ validation belongs to Phase 7

Phase 5A Design Principle

The curated layer uses logical primary keys and logical foreign keys.

Because Parquet does not enforce database constraints, the primary keys and foreign keys are documented and validated through lightweight sanity checks rather than enforced by the storage format.

Phase 5A Validation

The following validation commands were run successfully:

python -m py_compile spark\utils\model_config.py

python -c "import importlib.util;
spec=importlib.util.spec_from_file_location('model_config','spark/utils/model_config.py');
m=importlib.util.module_from_spec(spec); spec.loader.exec_module(m);
expected={'dim_customer','dim_account','dim_branch','dim_product','fact_transaction','fact_customer_risk_score','fact_daily_account_balance'}; actual=set(m.CURATED_MODEL_CONFIGS.keys()); assert actual==expected, f'Expected {expected}, got {actual}'; print('Curated model configs validated:', sorted(actual))"

Validation output:

Curated model configs validated: ['dim_account', 'dim_branch', 'dim_customer', 'dim_product', 'fact_customer_risk_score', 'fact_daily_account_balance', 'fact_transaction']

Phase 5A Commit

Commit:

2ef0791 Phase 5A - Define curated warehouse model design

---

6. Phase 5B: Build Curated Warehouse Models

Phase 5B Goal

The goal of Phase 5B was to implement the PySpark job that reads standardized Parquet inputs and writes curated warehouse-style Parquet outputs.

Phase 5B Files Changed

The following files were created or updated:

spark/jobs/build_curated_models.py

README.md

Phase 5B Work Completed

Phase 5B created the curated model build job:

spark/jobs/build_curated_models.py

The job reads standardized Parquet data from:

data/standardized/<table_name>/load_date=YYYY-MM-DD/<table_name>.parquet

It writes curated Parquet outputs to:

data/curated/<model_name>/load_date=YYYY-MM-DD/<model_name>.parquet

Phase 5B Job Capabilities

The curated build job supports:

- Reusable Spark session utility
- Reusable curated model configuration
- Load-date argument
- Input directory argument
- Output directory argument
- Log directory argument
- Optional model selection
- Load-date format validation
- Curated model whitelist validation
- Standardized Parquet reading
- Dimension model creation
- Fact model creation
- Curated Parquet writing
- Row-count sanity checks
- Duplicate logical key checks
- Null foreign key counts
- Join-loss tracking where applicable
- Summary CSV generation
- Non-zero failure behavior if any model fails

Phase 5B Command-Line Arguments

The job supports the following arguments:

--load-date

--input-dir

--output-dir

--log-dir

--models

Default input directory:

data/standardized

Default output directory:

data/curated

Default log directory:

logs/spark

---

7. Curated Model Design Summary

7.1 dim_customer

Purpose:

Supports customer-level reporting, customer governance, customer segmentation, and downstream data quality validation.

Grain:

One row per customer.

Logical primary key:

customer_id

Logical foreign keys:

branch_id -> dim_branch.branch_id

Source table:

customers

Final row count:

500

7.2 dim_account

Purpose:

Supports account-level analytics, account ownership analysis, product relationship analysis, and transaction relationship mapping.

Grain:

One row per account.

Logical primary key:

account_id

Logical foreign keys:

customer_id -> dim_customer.customer_id

product_id -> dim_product.product_id

branch_id -> dim_branch.branch_id

Source table:

accounts

Final row count:

990

7.3 dim_branch

Purpose:

Supports branch-level reporting, customer branch assignment, account branch assignment, and transaction branch analysis.

Grain:

One row per branch.

Logical primary key:

branch_id

Source table:

branches

Final row count:

10

7.4 dim_product

Purpose:

Supports banking product analysis, product ownership reporting, and account-product relationship mapping.

Grain:

One row per product.

Logical primary key:

product_id

Source table:

products

Final row count:

6

7.5 fact_transaction

Purpose:

Supports transaction-level analysis, financial activity reporting, reconciliation, fraud review, and future data quality validation.

Grain:

One row per transaction.

Logical primary key:

transaction_id

Logical foreign keys:

account_id -> dim_account.account_id

customer_id -> dim_customer.customer_id

branch_id -> dim_branch.branch_id

product_id -> dim_product.product_id

Source tables:

transactions

accounts

Design note:

fact_transaction joins transactions to accounts to bring account-related dimensional keys such as customer_id and product_id.

Final row count:

5000

Join-loss result:

0 records lost to join

7.6 fact_customer_risk_score

Purpose:

Supports customer risk monitoring, risk trend analysis, governance over risk-score fields, and future data quality validation.

Grain:

One row per customer risk score record.

Logical foreign key:

customer_id -> dim_customer.customer_id

Source table:

customer_risk_scores

Final row count:

742

7.7 fact_daily_account_balance

Purpose:

Supports account-day level balance summary analysis using available standardized account and transaction data.

Grain:

One row per account per balance date.

Logical composite key:

account_id

balance_date

Logical foreign keys:

account_id -> dim_account.account_id

customer_id -> dim_customer.customer_id

product_id -> dim_product.product_id

branch_id -> dim_branch.branch_id

Source tables:

accounts

transactions

Design note:

This model is a derived account-day summary based on available standardized data. It is not an official banking ledger balance table unless a true daily balance snapshot source is added in the future.

Final row count:

990

---

8. Phase 5B Validation Results

Python compilation passed.

Single model build validation passed:

dim_customer: 500 rows, 0 duplicate keys

Full curated model build succeeded for all seven models.

Full Build Results

Model	Rows	Duplicate Keys	Null FK Rows
dim_customer	500	0	0
dim_account	990	0	0
dim_branch	10	0	0
dim_product	6	0	0
fact_transaction	5,000	0	4,495
fact_customer_risk_score	742	0	0
fact_daily_account_balance	990	0	0

The fact_transaction null foreign key count includes optional fields such as branch_id and related_account_id. This is informational and was not treated as a Phase 5B failure.

The joined fact_transaction model produced:

5000 rows

0 records lost to join

This confirmed that transaction-to-account enrichment did not drop records.

---

9. Curated Output Location

The curated outputs were written using this pattern:

data/curated/<model>/load_date=2026-06-16/<model>.parquet

The seven curated outputs were created for:

data/curated/dim_customer/load_date=2026-06-16/dim_customer.parquet

data/curated/dim_account/load_date=2026-06-16/dim_account.parquet

data/curated/dim_branch/load_date=2026-06-16/dim_branch.parquet

data/curated/dim_product/load_date=2026-06-16/dim_product.parquet

data/curated/fact_transaction/load_date=2026-06-16/fact_transaction.parquet

data/curated/fact_customer_risk_score/load_date=2026-06-16/fact_customer_risk_score.parquet

data/curated/fact_daily_account_balance/load_date=2026-06-16/fact_daily_account_balance.parquet

These files are runtime outputs and are ignored by Git.

---

10. Summary CSV Output

Phase 5B generated a summary CSV under:

logs/spark/

Summary CSV generated:

logs/spark/curated_model_summary_20260618_151357.csv

The summary CSV includes model-level build evidence such as:

- Model name
- Status
- Source tables
- Output row count
- Duplicate key count
- Null foreign key count
- Input paths
- Output path
- Error message

This provides lightweight operational evidence for the curated model build.

---

11. Phase 5B Commit

Commit:

3fcb13d Phase 5B - Build curated warehouse models

Final Git status after Phase 5B:

clean

---

12. Git History After Phase 5

Latest Git history:

3fcb13d (HEAD -> main) Phase 5B - Build curated warehouse models

2ef0791 Phase 5A - Define curated warehouse model design

7cdf802 (tag: phase-4-complete) Phase 4B - Add schema handling for standardized Spark layer

9dde0d1 Phase 4A - Add PySpark raw to standardized processing

8554b4b (tag: phase-3-complete) Phase 3B - Harden raw ingestion validation

5a8e5d4 Phase 3A - Add PostgreSQL raw Parquet ingestion

Recommended Phase 5 tag:

phase-5-complete

---

13. Final Phase 5 Status

Phase 5 is complete.

Final Status

Item	Status
Phase 5A model design	Complete
Phase 5A model configuration	Complete
Data model documentation	Complete
Phase 5B curated build job	Complete
Curated dimensions	Complete

Item	Status
Curated facts	Complete
Standardized Parquet reading	Complete
Curated Parquet writing	Complete
Load-date partitioning	Complete
Model whitelist validation	Complete
Row-count sanity checks	Complete
Duplicate key checks	Complete
Basic null FK counts	Complete
Summary CSV reporting	Complete
README update	Complete
Git commits	Complete
Final working tree	Clean

---

14. Skills Practiced in Phase 5

Phase 5 helped practice the following skills:

- Data modeling
- Data warehousing
- Fact and dimension design
- Schema design
- Grain definition
- Logical primary key design
- Logical foreign key design
- Source-to-target thinking
- Curated warehouse layer design
- PySpark transformations
- PySpark joins

- Parquet data lake modeling
- Load-date partitioning
- Metadata-driven model configuration
- Row-count sanity checking
- Duplicate key checking
- Relationship sanity checking
- Documentation of business purpose, grain, keys, and relationships
- SDLC-style phased delivery
- Git commit discipline

---

15. Interview Explanation

A clear interview explanation for Phase 5 would be:

“In Phase 5, I designed and built the curated warehouse layer for the banking governance pipeline. The previous phase produced standardized Parquet outputs from raw banking source data. In Phase 5A, I defined the curated data model design, including dimensions, facts, grain, logical primary keys, logical foreign keys, source tables, target columns, and business purpose. I documented the design in a data model document and centralized the model definitions in a reusable configuration file.

In Phase 5B, I implemented a PySpark job to read standardized Parquet inputs and build seven curated warehouse models: customer, account, branch, and product dimensions, plus transaction, customer risk score, and daily account balance facts. The job writes curated Parquet outputs partitioned by load date and generates a summary CSV with row counts, duplicate key counts, null foreign key counts, input paths, output paths, and error messages. All seven models built successfully, transaction enrichment had zero join loss, and the working tree was clean after commit.

This phase gave me practical experience with warehouse-style fact and dimension modeling, schema design, grain definition, logical keys, PySpark joins, curated data layer creation, and documentation that supports future governance, cataloging, lineage, and data quality work.”

---

16. Phase 5 Conclusion

Phase 5 successfully created the curated warehouse layer for the Banking Data Governance & Data Quality Pipeline.

The project now has a working flow from:

PostgreSQL banking source system
→ Python ingestion layer

→ raw Parquet zone
→ PySpark standardized processing layer
→ standardized Parquet zone
→ curated warehouse fact and dimension layer

This prepares the project for the next phase:

Phase 6: SQL Data Quality Validation

Phase 6 should focus on SQL-based validation and reconciliation against the curated warehouse outputs. It should include null checks, duplicate checks, accepted-value checks, referential-integrity checks, source-to-target reconciliation, control totals, and exception-record queries.

PySpark data quality validation should remain separate for Phase 7.